



Project Report

Banking App

Generated by Raman Singh on January 29, 2026



Project Information

Banking App

Simple Banking App - API Microservice Showcase

This app simulates a lightweight digital banking platform built using a modular **microservice architecture**. It allows users to securely manage basic financial tasks through a clean API-driven backend.

Core Features

- **User Authentication:** Secure login and token-based session management
- **Account Overview:** View account details and balances
- **Transaction History:** Retrieve past transactions
- **Money Transfers:** Send funds to internal/external accounts with real-time balance validation
- **Audit Logging:** All sensitive actions are logged for traceability

Microservices

- **Auth Service** – Handles user login, registration, and token validation
- **Account Service** – Exposes account details and balances
- **Transaction Service** – Logs and fetches transaction history
- **Security Service** – Flags suspicious activity (optional)

Purpose

This app is designed as a demo project to **showcase API-first development, modular service boundaries, and testable backend logic** for modern financial applications.

Contents

Requirements	4
Requirements	4
R15 - Customer Service	4
R16 - Security Service	6
R17 - Account Service	6
R18 - Financial Transactions	9
R20 - Internal Money Movement	12
R21 - External Funds Transfer	13
Features	18
Features	18
F11 - Register Customer	18
F12 - Update Customer	18
F13 - Fetch Customer	18
F14 - Patch KYC Status	18
F15 - Customer Exists	19
F16 - Security	19
F17 - Exception handling	19
F18 - Observability	19
F19 - Configure Auth0- IDP	19
F20 - Provision User	20
F21 - Customer Login	20
F22 - Configure every microservice as resource server	20
F23 - Create a centralized security for all APIs	20
F24 - Create a User Account	20
F25 - Fetch Account details	20
F26 - Fetch Account Balance	21
F27 - Allow an admin to update an account	21
F28 - Place Hold on an account	21
F29 - Release hold	21
F30 - Credit account	22
F31 - Debit account	22
F32 - Find all Account Transactions associated with an account	22
F33 - Every Debit Must Record its associated transaction	22
F34 - Every Credit Must Record its associated transaction	22
F35 - Every Hold Must Record its associated transaction	22
F36 - Every 'Hold Release' Must Record its associated transaction	22

Requirements

Requirements

R15 - Customer Service

Create Customer — Use Case

Goal

Allow a person to create a customer profile

Who

- **Customer (new user)**
- **Operations** (manual KYC approver, if needed)

Triggers

Customer selects “Sign up” in the app and submits updated information.

Happy Path

1. Customer enters first name, last name, email, phone, and address.
2. System checks that the email/customerId isn't already used.
3. System creates a customer profile with **KYC Pending**.

Variations / Edge Cases

- **Duplicate email**
- **Duplicate externalId**

Business Rules

- Email must be unique.
- Address is mandatory for KYC.
- **Login is disabled until KYC Verified.**

Acceptance Criteria

- Given a new email, when I submit the form, then I see a success message and my status is **KYC Pending**.
- Given **KYC Verified**, when I return, then I can sign in successfully.

Update Customer

Goal

Allow an existing customer to update their profile details (e.g., phone, address, nickname).

Who

- Customer (logged-in user)
- System (validates changes)

Triggers

Customer selects “Update Profile” in the app and submits updated information.

Happy Path

- Customer provides new details (e.g., address change).
- System validates the fields.
- System updates the record in the customers table and bumps the version number.
- Customer sees: “Your profile has been updated successfully.”
-

Variations / Edge Cases

- Attempt to change immutable fields (e.g., externalId, email). → Error “Field not updatable.”

Business Rules

- Certain fields are immutable (externalId).
- Optimistic locking via version to prevent overwrite.

Acceptance Criteria

- Given valid updates, when I submit, then my profile reflects the new details.
- Given concurrent update, when version mismatch occurs, then I get a 409 Conflict.

Update Customer KYC

Goal

Allow operations to update customer's KYC .

Who

System

Happy Path

4. KYC completes (automated or operations approves it).
5. System marks the profile **KYC Verified** and creates login credentials.
6. Customer can log in.

Variations / Edge Cases

- **KYC failed** → status becomes **KYC Failed**;

Acceptance Criteria

Given **KYC Verified**, when I return, then I can sign in successfully.

Get Customer — Use Case**Goal**

Retrieve customer details for display in the app or internal operations.

Who

- Customer (self-view)
- Operations (support staff with scope)

Triggers

User logs in and navigates to profile, or support queries customer details.

Happy Path

- System fetches the customer record by ID/externalId.
- Sensitive fields (like email, phone) shown only if scope permits.
- Customer sees their details in the UI.

Variations / Edge Cases

- Invalid ID → 404 Not Found.
- Customer inactive → "Profile not available."

Business Rules

- Data returned must exclude sensitive audit fields (version, internal IDs).

Acceptance Criteria

- Given a valid ID, when I call Get Customer, then I see profile data.
- Given an invalid ID, when I call Get Customer, then I see 404 Not Found.
- Given insufficient scope, when I call Get Customer, then I see 403 Forbidden.

Customer Exists — Use Case**Goal**

Check if a customer exists (by email or externalId) before creating or linking accounts.

Who

- Account Service, Payment Service, or any upstream system needing validation.

Triggers

External system calls "/customers/exists?email=..." or "/customers/exists?externalId=..."

Happy Path

- System searches the customer table by provided key.
- Returns boolean: true if found, false if not.

Variations / Edge Cases

- Invalid format → 400 Bad Request.

Business Rules

- Must be idempotent and side-effect free.
- Should not leak sensitive data beyond true/false.

Acceptance Criteria

- Given existing email, when I call exists, then I get true.
- Given non-existing email, when I call exists, then I get false.
- Given invalid parameter, when I call exists, then I get 400 error.

State: Satisfied (locked)

Priority: Must Have

R16 - Security Service

AuthUser Use Case

Goal

Once a customer completes KYC, they should receive login credentials and be able to sign in via Auth0. Upon successful login, they receive a **JWT access token** containing the correct **roles and scopes** to securely access backend APIs.

Internal backend services should also be able to request **M2M tokens** using the client_credentials flow when calling each other.

Actors

- Customer (end user using SPA frontend)
- **AuthUser Service** – our backend service that provisions users in Auth0
- **Customer Service** – marks customers as KYC verified
- **Auth0** – the Identity Provider (IdP)
- **Backend Services** – e.g., Payment Orchestrator calling Transaction service (M2M clients)

Preconditions

- Customer's KYC status is VERIFIED
- Auth0 is configured with:
 - Applications: SPA (PKCE) and M2M (client_credentials)
 - API: MockBank API with audience https://mockbank/api
 - RBAC enabled + "Add Permissions in Access Token" enabled
 - Roles: ROLE_CUSTOMER, ROLE_OPS, ROLE_SERVICE
 - Permissions/scopes like fdx:accounts.read, fdx:transactions.write, etc.
 - Post-login Action to add **custom claims** (e.g., role, customer_id)

Primary Flow (Happy Path)

1. **AuthUser Service** provisions the user in Auth0
 - Calls Auth0 **Management API** to create user (if not already existing)
 - Assigns the role (ROLE_CUSTOMER) and required permissions
 - Optionally triggers a password setup or verification email
2. **Customer logs in (SPA)**
 - SPA redirects to Auth0 **Universal Login**
 - Authorization Code + **PKCE** flow executes
 - SPA receives an **Access Token (JWT)** and optional **Refresh Token**
3. **Accessing Resource Servers**
 - SPA calls secure APIs with Authorization: Bearer <token>
 - Resource Server validates the JWT (signature, audience, scopes) and authorizes access
4. **Service-to-Service (M2M) Flow**
 - Backend service uses client_credentials grant to get a token from Auth0
 - Token is cached and reused until it expires
 - Used for calls like Payment Orchestrator → Transaction Service

⚠ Edge Cases

- **Expired/invalid token** → Resource returns 401/403; client refreshes token

State: Satisfied (locked)**Priority:** Must Have

R17 - Account Service

Create Account — Use Case

Goal

Allow a customer to open a new bank account (e.g., Chequing, Savings).

Who

- Customer (logged-in user)
- System (validates & creates account)

Triggers

Customer selects **“Open New Account”** in the app and submits required information.

Happy Path

1. Customer chooses account type, currency, and initial preferences.
2. System validates that the customer is active and KYC Verified.
3. System creates the account with **status = ACTIVE**, initial balance = 0.
4. System returns the new account details.

Variations / Edge Cases

- Customer not KYC Verified → show “KYC must be completed before opening account.”
- Customer already has max allowed accounts of same type (business rule).

Business Rules

- Customer must be **KYC Verified** to open an account.
- Account starts with **0 balance** and **version = 1**.
- Currency must be supported (e.g., CAD).
- Idempotency-Key required to avoid duplicate account creation on retry.

Acceptance Criteria

- Given a KYC Verified customer, when I open a new account, then the account is created with status ACTIVE.
- Given I retry the same request with the same Idempotency-Key, then I receive the same response, not a duplicate account.
- Given I am not KYC Verified, when I attempt to create an account, then I see an error and account is not created.

Get Account — Use Case

Goal

Retrieve account details for display in the app.

Who

- Customer (viewing their own account)
- Operations (support staff)

Triggers

User selects an account from the app dashboard.

Happy Path

5. System fetches the account by accountId.
6. System returns account details including balances and status.

Variations / Edge Cases

- Account does not belong to user → 403 Forbidden.
- Account not found → 404 Not Found.

Business Rules

- Only the owner or authorized staff can view account details.
- ETag should be included for concurrency control on future updates.

Acceptance Criteria

- Given a valid account I own, when I view details, then I see account info and latest balances.
- Given an invalid account ID, then I receive 404.
- Given I try to access someone else’s account, then I receive 403 Forbidden.

List Accounts for Customer — Use Case

Goal

Allow a customer to view all their linked accounts.

Who

- Customer

Triggers

User opens “My Accounts” screen.

Happy Path

7. System fetches all accounts linked to the logged-in customer.
8. Accounts appear in the app.

Variations / Edge Cases

- Customer has no accounts → show empty list with “No accounts found”.

Business Rules

- Only show accounts of the logged-in customer.
- Joint/secondary accounts support can be added later.

Acceptance Criteria

- Given I am a logged-in customer, when I open My Accounts, then I should see a list of all my accounts.
- Given I have no accounts, when I open My Accounts, then I see an empty list.

Update Account Status — Use Case**Goal**

Enable operations to change account status (e.g., Suspend or Close account).

Who

- Operations (authorized user)
- System

Triggers

Fraud alert, compliance reason, or account closure request.

Happy Path

9. Operations selects an account and chooses a status update (e.g., ACTIVE → SUSPENDED).
10. System validates the transition.
11. System updates status and records audit log.

Variations / Edge Cases

- Status change not allowed (e.g., CLOSED → ACTIVE).
- Close account requested but balance not zero → reject with reason.

Business Rules

- Account cannot be closed unless balance = 0 and no active holds exist.
- Status transitions must follow allowed state diagram.
- Only authorized roles can perform this action.

Acceptance Criteria

- Given a valid transition (ACTIVE → SUSPENDED), then the status updates successfully.
- Given an invalid transition (CLOSED → ACTIVE), then I receive a validation error.
- Given a close request with non-zero balance, then system rejects the closure.

Place Funds Hold — Use Case**Goal**

Reserve funds on an account for a pending payment (e.g., bill payment or RTR).

Who

- Payments Orchestrator (system)

Triggers

A payment request is initiated requiring funds reservation.

Happy Path

12. System checks available balance.
13. System creates a funds hold and reduces available balance.
14. System returns hold reference ID.

Variations / Edge Cases

- Insufficient balance → hold fails.
- Duplicate hold retry with same Idempotency-Key → return existing hold.

Business Rules

- Held funds reduce available balance but not current balance.
- Holds are temporary and must be released or posted later.

Acceptance Criteria

- Given sufficient funds, when a hold is placed, then available balance decreases.
- Given insufficient funds, then hold request is rejected.
- Given same Idempotency-Key retry, then same hold response is returned.

Release Funds Hold — Use Case**Goal**

Free up previously reserved funds.

Who

- Payments Orchestrator (system)
- Operations (manual override)

Triggers

Payment cancelled or completed.

Happy Path

15. System finds active hold.
16. System marks hold released.
17. Available balance increases accordingly.

Variations / Edge Cases

- Hold already released → return idempotent success.

- Hold not found → 404.

Business Rules

- Holds can only be released once.

Acceptance Criteria

- Given valid hold, when I release it, then the held amount becomes available.
- Given a second release attempt, then no error and no double credit occurs.

Credit Account — Use Case

Goal

Add money into a customer account.

Who

- System (payments, transfers, refunds)

Triggers

Incoming transfer, deposit, or completed payment posting.

Happy Path

18. System validates request & ETag (if provided).
19. Amount is added to current balance and version increments.
20. Updated account details return to caller.

Variations / Edge Cases

- Stale ETag version → reject with 409 Conflict.
- Currency mismatch → reject.

Business Rules

- Must be idempotent at posting level (correlationId recommended).

Acceptance Criteria

- Given a valid request, then balance increases and new version returns.
- Given a stale version, then I receive a 409 Conflict.

Debit Account — Use Case

Goal

Deduct money from a customer account.

Who

- System (payments, transfers)

Triggers

Payment posting or outgoing transfer.

Happy Path

21. Validate sufficient available balance.
22. Debit amount and bump version.
23. Updated account details returned.

Variations / Edge Cases

- Insufficient balance → reject.
- Stale ETag → 409 Conflict.

Business Rules

- Cannot debit more than available balance.
- Must support optimistic locking to prevent double debit.

Acceptance Criteria

- Given sufficient balance, when debit occurs, then balance reduces and version increases.
- Given insufficient balance, then debit fails with appropriate message.
- Given concurrent debit, then one succeeds and others receive 409 Conflict.

State: In Scope (locked)

Priority: Must Have

R18 - Financial Transactions

Use Case: Record Financial Transactions for Every Account Posting

1. Summary

Whenever money moves on an account (hold, release, debit, credit), the system must:

1. Update the **account state** (balance or holds), and
2. Persist a corresponding **financial transaction record**

Both operations must run in a **single database transaction** so that they either **succeed together or fail together** (all-or-none). This ensures the account balance and financial history never go out of sync.

2. Goal / Business Value

- Maintain a **trustworthy ledger** for every customer account.
- Ensure **balances and transactions are always consistent**.
- Provide a reliable source for:
 - customer statements
 - audit and compliance
 - dispute resolution
 - downstream reporting and analytics

3. Scope

This use case is **inside the Account Service** and covers:

- **Account balance updates** (debit and credit)
- **Account holds** (place and release)
- **Financial transaction entries** linked to each posting/hold event

Out of scope (for now):

- Separate Transaction microservice
- Saga / Outbox / Kafka-based distributed consistency
 - These will be addressed later in the **Payments / Event-Driven** flows.

4. Actors

- **API Client (Frontend / External Integration)**
 - Calls the Account Service APIs (debit, credit, hold, release).
- **Account Service**
 - Validates requests, enforces business rules, updates account/hold tables.
- **Financial Transaction Component (within Account Service)**
 - Persists financial transaction records linked to each operation.
- **Identity Provider (Auth0)**
 - Issues access tokens with required scopes (fdx:accounts.read, fdx:accounts.write, admin scopes).
- **Database (Accounts DB)**
 - Stores:
 - account
 - account_hold
 - account_transaction (or similar)

5. Preconditions

- Customer account exists and is in a valid status for the requested operation (e.g., not closed, not fully blocked, holds allowed if account is active).
- Caller is authenticated and authorized:
 - Normal users: appropriate FDX scopes (e.g., fdx:accounts.write).
 - Admin/ops: admin scopes for status changes or special adjustments.
- For debit/credit operations, caller provides:
 - a valid **ETag** (version) via If-Match header
 - optional **idempotency key** for safe retries

6. Triggers

This use case is triggered whenever one of the following APIs is called:

3. **Place Hold** – POST /api/v1/accounts/{id}/holds
4. **Release Hold** – POST /api/v1/accounts/{id}/holds/{holdId}/release
5. **Debit Account** – POST /api/v1/accounts/{id}/debit
6. **Credit Account** – POST /api/v1/accounts/{id}/credit

7. Main Success Scenario (High-Level)

For each of the four operations, the pattern is:

7. **Validate request & authorization**
 - Verify JWT, scopes, and ownership/admin rules.
 - Validate that the account is in a valid state for the requested operation.
8. **Load account (and hold if applicable)**
 - Fetch the account record by ID.
 - For debit/credit, check the **ETag/version** and reject if it doesn't match.
 - For hold/release, fetch or create the hold record as needed.
9. **Apply business rules**
 - For **hold**: compute available balance (ledger – active holds), ensure enough funds.
 - For **debit**: compute available balance and ensure funds are sufficient.
 - For **credit**: validate that credit is allowed for account type.
 - For **release**: ensure the hold is active and belongs to the account.

10. Update account / hold data

- Debit: reduce ledger balance.
- Credit: increase ledger balance.
- Place Hold: insert a row into account_hold with status = ACTIVE.
- Release Hold: update account_hold.status to RELEASED (or equivalent).

11. Create financial transaction record

- In the same service method, build a Transaction entity that includes:
 - accountId
 - amount
 - currency
 - type (DEBIT, CREDIT, HOLD_PLACED, HOLD_RELEASED, etc.)
 - reason / description
 - correlationId / traceId (for observability)
 - idempotency fingerprint (if provided)
 - optional balanceAfter for postings (debit/credit)
 - timestamp
- Save this transaction using the same datasource and within the same Spring @Transactional boundary.

12. Commit or Rollback

- If both:
 - account/hold changes, **and**
 - transaction insert
 - succeed → commit the DB transaction.
- If any part fails (e.g., constraint violation on transaction insert, DB error, unexpected exception):
 - the entire transaction is rolled back →
 - **no partial balance change, no orphan transaction row.**

13. Return response

- Return updated DTOs back to the client (e.g., updated account, hold status, etc.).
- The ETag in the response reflects the new account version.

8. Alternate / Error Flows

14. Insufficient Available Funds (Hold or Debit)

- If requestedAmount > availableBalance:
 - Do not update account or holds.
 - Do not create a transaction record.
 - Return an error (e.g., 400 Bad Request with message “Insufficient available funds”).

15. ETag Mismatch (Debit/Credit)

- If If-Match version ≠ current account version:
 - No update is performed.
 - No transaction record is created.
 - Return 409 Conflict (version mismatch).

16. Idempotency Replay (Hold / Transaction)

- If a request with the same idempotency key / fingerprint is seen again:
 - Detect existing hold or transaction record.
 - Return the existing result instead of creating a duplicate.
 - No new account/hold/transaction rows should be created.

17. Transaction Insert Fails

- If the account_transaction insert fails for any reason (e.g., constraint, DB error, bug):
 - The whole database transaction is rolled back.
 - The account balance and holds revert to their previous state.
 - The client receives an error; no partial updates are visible.

18. Unauthorized Access

- If token is missing or lacks required scope:
 - Request is rejected before any DB interaction.
 - No state is changed, no transaction recorded.

9. Business Rules / Invariants

- **Balance & transaction history must never diverge.**
- There is no valid state where balance is changed but no transaction exists, or vice versa.
- **One posting = one transaction entry.**
- Every debit/credit should be traceable via a corresponding transaction row.
- **Available balance calculation**
 - Available = Ledger – Sum of active holds.
- **Idempotent behavior** for external calls:

- Same idempotency key + same payload → same result, no new rows created.
- **Strict ownership and scope checks**
 - Customers can only act on their own accounts with correct scopes.
 - Admin endpoints are restricted to admin scopes.

State: In Scope (locked)

Priority: Must Have

R20 - Internal Money Movement

Internal Money Movement - Use Case (via Payment Orchestrator)

Goal: Move money between two internal accounts (e.g. Chequing → Savings) using the same **Payment Orchestrator** and **payments** table that BillPay uses.

1. API - Create Internal Transfer

Endpoint

POST /api/v1/payments/internal-transfer

Authorization: Bearer <JWT>

Idempotency-Key: <uuid>

Content-Type: application/json

Request body

```
{
  "fromAccountId": "11111111-1111-1111-1111-111111111111",
  "toAccountId": "22222222-2222-2222-2222-222222222222",
  "amount": 250.00,
  "currency": "CAD",
  "executionDate": "2025-12-06",
  "reference": "Move to savings"
}
```

202 Accepted response

```
{
  "paymentId": "6c5e1f4c-5f9a-4c7b-b9ac-4e4e91cfb001",
  "state": "FUNDS_HELD"
}
```

2. What Payment Orchestrator does

When /internal-transfer is called:

1. **Validate accounts & limits** via AccountService (both accounts belong to same customer, sufficient balance, etc.).
2. Ask AccountService to **place a hold** on fromAccountId for the amount.
3. Insert a row into the payments table:
 - paymentId – new UUID
 - state – FUNDS_HELD
 - debtorAccountId – fromAccountId
 - amountValue / amountCcy – from request
 - executionDate – from request
 - invoiceReference – client reference ("Move to savings" or a generated ref)
 - billerRefNumber – can be reused to store toAccountId or left for later refactor
 - batchId – null (no batch for internal)
 - externalStatusCode – null initially
 - reason – "Funds held for internal transfer"
 - idempotencyKey, createdAt, updatedAt – as usual
4. Publish a Kafka event:

Topic: internal.payment.requested

Producer: Payment Orchestrator

```
{
  "paymentId": "6c5e1f4c-5f9a-4c7b-b9ac-4e4e91cfb001",
  "fromAccountId": "1111...",
  "toAccountId": "2222...",
  "amount": 250.00,
  "currency": "CAD"
}
```

Business state at this point: **FUNDS_HELD**.

3. Internal Money Movement Worker

Service: InternalMoneyMovementWorker

Consumer of: internal.payment.requested

For each event:

5. **Consume hold + debit** the source account
 - e.g. POST /accounts/{from}/capture-hold
6. **Credit** the destination account
 - POST /accounts/{to}/credit
7. On full success, publish:

// topic: internal.payment.status

```
{
  "paymentId": "6c5e1f4c-5f9a-4c7b-b9ac-4e4e91cfb001",
  "status": "POSTED",
  "reason": "Internal transfer completed"
}
```

8. On any failure (after retries/compensation), publish:

```
{
  "paymentId": "6c5e1f4c-5f9a-4c7b-b9ac-4e4e91cfb001",
  "status": "FAILED",
  "failureCode": "DEBIT_FAILED | CREDIT_FAILED | COMPENSATION_FAILED",
  "reason": "Account service timeout"
}
```

Producer: InternalMoneyMovementWorker

Topic: internal.payment.status

4. Orchestrator - Final State Update

Payment Orchestrator also **consumes** internal.payment.status:

- If status = POSTED
 - update payments.state = POSTED
 - set reason = "Internal transfer posted"
- If status = FAILED
 - update payments.state = FAILED
 - set externalStatusCode / reason from event

So for internal money movement the lifecycle in the payments table is:

FUNDS_HELD → POSTED (happy path)

FUNDS_HELD → FAILED (error path)

State: In Scope (locked)

Priority: Must Have

R21 - External Funds Transfer

External Funds Transfer (EFT) - Requirements Pack

(External Account Service + EFT Saga using Payment Orchestrator)

Goal

Enable customers to send money from an internal account to an **external bank account** using:

- **Payment Orchestrator** (same service + payments table as BillPay)

- **Kafka events + Saga pattern**
- **Outbox pattern**
- **Idempotency + retries**
- **M2M tokens** for async service-to-service calls

A) External Account Service (Beneficiaries)

A1. API - Add External Account

POST /api/v1/external-accounts

Headers:

- Authorization: Bearer <JWT>
- Idempotency-Key: <uuid>

Body:

```
{
  "nickname": "My RBC Chequing",
  "beneficiaryName": "Raman Singh",
  "bankCode": "003",
  "transit": "12345",
  "accountNumber": "9876543",
  "accountType": "CHEQUING",
  "currency": "CAD"
}
```

Response **201 Created**

```
{
  "externalAccountId": "a2ad7c2f-8b1f-4b93-a50b-2f0b3d2d9e11",
  "status": "ACTIVE"
}
```

A2. API - List External Accounts

GET /api/v1/external-accounts

Header: Authorization: Bearer <JWT>

Response:

```
{
  "items": [
    {
      "externalAccountId": "a2ad7c2f-8b1f-4b93-a50b-2f0b3d2d9e11",
      "nickname": "My RBC Chequing",
      "beneficiaryName": "Raman Singh",
      "bankCode": "003",
      "transit": "12345",
      "accountNumberMasked": "****543",
      "accountType": "CHEQUING",
      "currency": "CAD",
      "status": "ACTIVE"
    }
  ]
}
```

A3. API - Deactivate External Account (Delete)

Prefer **soft delete**.

DELETE /api/v1/external-accounts/{externalAccountId}

Header: Authorization: Bearer <JWT>

Response: **204 No Content**

Behavior:

- Mark status = INACTIVE
- Prevent this external account from being used in new EFT requests

A4. Service Rules

- External accounts are **owned by customerId** from JWT
- Validate format of bankCode/transit/accountNumber
- Store **masked account number** for UI display

- (Open discussion) Encrypt account number at rest

B) EFT Payment - Use Case via Payment Orchestrator

B1. API - Create EFT Transfer

POST /api/v1/payments/eft

Headers:

- Authorization: Bearer <JWT>
- Idempotency-Key: <uuid>

Body:

```
{
  "fromAccountId": "11111111-1111-1111-1111-111111111111",
  "externalAccountId": "a2ad7c2f-8b1f-4b93-a50b-2f0b3d2d9e11",
  "amount": 500.00,
  "currency": "CAD",
  "executionDate": "2025-12-07",
  "reference": "Rent payment"
}
```

Response **202 Accepted**

```
{
  "paymentId": "9a7b21aa-3c2d-4f9b-9d2a-8bfa73c1c999",
  "state": "FUNDS_HELD"
}
```

B2. What Payment Orchestrator Does (Single Transaction)

When /payments/eft is called:

1. **Idempotency**
 - Lookup payment by idempotencyKey
 - If exists → return existing result (no duplicates)
2. **Validation**
 - Validate amount/currency/date
 - Validate fromAccountId belongs to customer
 - Call **ExternalAccountService** to ensure:
 - externalAccount belongs to customer
 - status is ACTIVE
 - fetch beneficiary details
3. **Funds Hold**
 - Call AccountService to place hold on fromAccountId
 - store holdId
4. **Persist Payment + Outbox**
5. Insert payment row:
 - state = FUNDS_HELD
 - channel = EFT
 - debtorAccountId = fromAccountId
 - amountValue / currency / executionDate
 - reference
 - externalAccountId
 - holdId
 - batchId = null initially
 - idempotencyKey, timestamps, etc.

Write Outbox event:

- EftPaymentRequested

Outcome: money reserved + payment stored + event staged.

B3. Kafka Event - EFT Requested

Topic: eft.payment.requested

Producer: Payment Orchestrator

```
{
```

```

"eventId": "...",
"paymentId": "9a7b21aa-3c2d-4f9b-9d2a-8bfa73c1c999",
"fromAccountId": "1111...",
"externalAccountId": "a2ad...",
"amount": 500.00,
"currency": "CAD"
}

```

C) EFT Worker - Batching + File Preparation

C1. Service

EftWorkerService

Consumes: eft.payment.requested

For each event:

- Find/create an OPEN batch (daily or threshold-based)
- Add BatchLine (batchId, paymentId, lineNo)
- Emit EftPaymentEnqueued so Orchestrator can mark payment as BATCHED (optional but consistent with BillPay)
- When batch reaches threshold, emit EftBatchReady

Topic: eft.batch.ready

D) Settlement Service - Submit EFT Batch + Retry

Consumes: eft.batch.ready

Responsibilities:

- Create settlement record (status READY, retryCount=0)
- Generate file if needed (EFT file format mock acceptable)
- Upload to external clearing system (simulated)
- On success → emit EftBatchSubmitted
- On failure → mark failed, increment retryCount, emit retry event:
 - eft.batch.retry
- After max retries → emit DLQ-style event:
 - eft.batch.dlq

E) EFT Acknowledgements - Final Payment Status

Since no real clearing exists:

- Provide a mock ack flow (can be REST-triggered like you did for BillPay)

Publish per-payment status:

Topic: eft.payment.status

```

{
  "paymentId": "9a7b21aa-3c2d-4f9b-9d2a-8bfa73c1c999",
  "status": "POSTED | FAILED",
  "failureCode": "...",
  "reason": "Accepted by clearing / rejected / timeout"
}

```

F) Orchestrator - Finalize the Payment

Consumes: eft.payment.status

If POSTED:

- Call AccountService:
 - releaseHold
 - debitAccount
- Update payment state = POSTED

If FAILED:

- releaseHold
- Update payment state = FAILED

Also record processed_events for event-level idempotency.

G) Security Requirement (Important)

Kafka listeners won't have user JWT.

Any async service-to-service call (Settlement/Orchestrator → AccountService) must use:

- **Auth0 M2M (client_credentials)**

- Feign RequestInterceptor injecting:
 - Authorization: Bearer <m2m token>

Add scope like:

- admin:accounts

H) Payment Lifecycle Summary

Happy path:

FUNDS_HELD → BATCHED → SUBMITTED → POSTED

Failure path:

FUNDS_HELD → FAILED (and hold released)

State: In Scope (locked)

Priority: Must Have

Features

Features

F11 - Register Customer

Allow a new customer to be created with mandatory fields (firstname ,last name , email , address , phone and customer Id)

Acceptance Criteria

- 1 Request with valid fields creates a new row in the customers table.
- 2 Duplicate Email - should get 409 error
- 3 Duplicate customerID - again 409 error
- 4 Respose should return a 201 created .

State: Done (locked)

Finish By: 2025-09-30

F12 - Update Customer

Allow an existing customer to update their profile details (e.g., phone, address, nickname).

- Acceptance Criteria:
 - i. Request with correct ETag updates record, increments version.
 - ii. Request with stale ETag returns 412 Precondition Failed.
 - iii. Response returns updated resource with new ETag.
 - iv. Update to customerId should not be allowed

State: Done (locked)

Finish By: 2025-09-30

F13 - Fetch Customer

Description: Retrieve an existing customer by externalId.

- Acceptance Criteria:
 - i. Valid ID returns customer record.
 - ii. Non-existing ID returns 404 Not Found.
 - iii. Response includes ETag header.

State: Done (locked)

Finish By: 2025-09-30

F14 - Patch KYC Status

- Allow KYC status update (PENDING → VERIFIED or REJECTED).
- Acceptance Criteria:
 - i. **KYC Verified**, when I return, then I can sign in successfully. Create Login Credentials for the customer

State: Done (locked)
Finish By: 2025-09-12

F15 - Customer Exists

Check if a customer exists (by email or externalId) before creating or linking accounts.

Acceptance Criteria

- Given existing email, when I call exists, then I get true.
- Given non-existing email, when I call exists, then I get false.
- Given existing customerID, when I call exists, then I get true.
- Given non-existing customerID, when I call exists, then I get false.

State: Done (locked)
Finish By: 2025-09-30

F16 - Security

Secure relevant endpoints

State: Done (locked)
Finish By: 2025-09-30

F17 - Exception handling

Enable centralized exception handling

State: Done (locked)
Finish By: 2025-09-30

F18 - Observability

• Logging & Audit

- Acceptance Criteria:
 - i. All API calls log with X-Request-ID (correlation ID).

State: Done (locked)
Finish By: 2025-09-30

F19 - Configure Auth0- IDP

- Auth0 is configured with:
 - Applications: SPA (PKCE) and M2M (client_credentials)
 - API: MockBank API with audience https://mockbank/api
 - RBAC enabled + "Add Permissions in Access Token" enabled
 - Roles: ROLE_CUSTOMER, ROLE_OPS, ROLE_SERVICE
 - Permissions/scopes like fdx:accounts.read, fdx:transactions.write, etc.
 - Post-login Action to add **custom claims** (e.g., role, customer_id)

State: Done (locked)
Finish By: 2025-09-28

F20 - Provision User

provision the user in Auth0

- Calls Auth0 **Management API** to create user (if not already existing)
- Assigns the role (ROLE_CUSTOMER) and required permissions

State: Done (locked)

Finish By: 2025-09-28

F21 - Customer Login

1. Customer logs in (SPA)

- SPA redirects to Auth0 **Universal Login**
- Authorization Code + **PKCE** flow executes
- SPA receives an **Access Token (JWT)** and optional **Refresh Token**

State: Done (locked)

Finish By: 2025-09-28

F22 - Configure every microservice as resource server

1. Service-to-Service (M2M) Flow

- Backend service uses client_credentials grant to get a token from Auth0
- Token is cached and reused until it expires

State: Done (locked)

Finish By: 2025-09-28

F23 - Create a centralized security for all APIs

All microservices should be a resource Server

IDP- Auth0

Establish integration between Auth0 and all Microservices

State: Done (locked)

Finish By: 2025-10-03

F24 - Create a User Account

Allow a customer to open a new bank account (e.g., Chequing, Savings).

State: Done (locked)

Finish By: 2025-11-12

F25 - Fetch Account details

Fetch account details of the customer / customers

Mind the scopes - Admins have different scopes than users

User scopes

[fdx:accounts.write](#)

[fdx:accounts.read](#)

Admin scopes

[admin:accounts.read](#)

[admin:accounts.write](#)

State: Done (locked)

Finish By: 2025-11-16

F26 - Fetch Account Balance

An API to fetch user balance

The API should return

" Toral balance"

"Total Funds on Hold"

"Available Balance "

State: Done (locked)

Finish By: 2025-11-16

F27 - Allow an admin to update an account

Only an admin can freeze or close an account

mind the scopes

Token should have this scope to enable this operation

Scope - [admin:accounts.write](#)

State: Done (locked)

Finish By: 2025-11-16

F28 - Place Hold on an account

Ability to place a hold on funds

State: Done (locked)

Finish By: 2025-11-18

F29 - Release hold

Ability to Release hold

State: Done (locked)

Finish By: 2025-11-18

F30 - Credit account

Credit account

State: Done (locked)

Finish By: 2025-11-18

F31 - Debit account

Debit account

State: Done (locked)

Finish By: 2025-11-18

F32 - Find all Account Transactions associated with an account

The customer should be able to fetch all transactions related his account

State: To Do (locked)

Finish By: 2025-11-22

F33 - Every Debit Must Record its associated transaction

Every Debit Must Record its associated transaction

Make Sure it follows DB transaction - All or none ie debit account and record transaction

State: Done (locked)

Finish By: 2025-11-22

F34 - Every Credit Must Record its associated transaction

Every Credit Must Record its associated transaction

Make Sure it follows DB transaction - All or none ie debit account and record transaction

State: Done (locked)

Finish By: 2025-11-22

F35 - Every Hold Must Record its associated transaction

State: Done (locked)

Finish By: 2025-11-22

F36 - Every 'Hold Release' Must Record its associated transaction

State: Done (locked)

Finish By: 2025-11-22